Launch Profiles Guide: VS Code and WinUAE Machine Profiles

A companion guide to Building an AMIGA Game Engine

Richard Shannon

AmigaGameEngine v1.0

Contents

1. The Profiles 1

2. Settings by Profile 2

3. What the Settings Mean 2

4. The Build Task 3

The repository's `.vscode/launch.json` defines eight profiles: six WinUAE profiles, one for each kind of machine the engine must run on, a WinUAE profile that boots Workbench with the game's disk in `DF1:`, and an FS-UAE profile for macOS and Linux. Pick one in VS Code's Run and Debug panel and press F5: the build task assembles the game, WinUAE starts with that machine's settings, and the debugger stops on breakpoints in your source.

Chapter 4 explains how to install the toolchain and the amiga-assembly extension, and how to point the profiles at your own Kickstart ROMs. This guide lists what each profile emulates, what every setting means, and what the build task does.

1. The Profiles

Profile	Machine	Why test on it
A500 PAL	68000, OCS, 512 KB Chip + 512 KB slow, Kickstart 1.3	the baseline: slowest CPU, oldest OS, smallest Chip RAM
A500 AROS PAL	the same machine, with the free AROS ROM	runs without an original Kickstart ROM
A500+ NTSC	68000, ECS, 1 MB Chip, Kickstart 2.04	60 Hz timing and the shorter NTSC frame
A600 PAL	68000, ECS, 1 MB Chip, Kickstart 2.05	the last 68000 Amiga: ECS at 50 Hz with Chip RAM only, Kickstart 2.x, and Gayle's IDE and PCMCIA hardware
A1200 PAL	68020, AGA, 2 MB Chip, Kickstart 3.1	AGA registers (`FMODE`, `BPLCON3/4`), CPU cache
A4000 PAL	68040, AGA, 2 MB Chip + 4 MB Fast, Kickstart 3.1	a CPU far ahead of the chipset: exposes timing and cache assumptions
A500 Workbench 1.3 (Floppy) + DH0 PAL	the A500 PAL machine, booting Workbench 1.3 from `adfs/amiga-os-134-workbench.adf` in `DF0:`, with `adfs/game.adf` in `DF1:`	starting the game from its own icon, with Workbench holding Chip RAM (Chapter 32 of the book)
FS-UAE A500 PAL	the A500 PAL machine in FS-UAE: 512 KB Chip + 512 KB slow, Kickstart 1.3, `uae/dh0` as the hard drive	WinUAE runs only on Windows; FS-UAE runs on macOS and Linux too

No Kickstart ROM? The AROS profile uses WinUAE's built-in AROS ROM replacement (`kickstart_rom_file=:AROS`), which is free and needs no download. The game runs on it, so you can build and test everything in this repos-

itory before sourcing an original ROM. Keep the Kickstart profiles for final compatibility testing: AROS is a reimplementation, not the original OS.

2. Settings by Profile

Setting	A500	A500 AROS	A500+	A600	A1200	A4000
`kickstart_rom_file`	`kick1.3.rom`	`:AROS`	`kick2.04.rom`	`kick2.05.rom`	`kick3.1.rom`	`kick3.1.rom`
`chipset`	`ocs`	`ocs`	`ecs`	`ecs`	`aga`	`aga`
`chipset_compatible`				`A600`		
`chipmem_size`	`1`	`1`	`2`	`2`	`4`	`4`
`bogomem_size`	`2`	`2`	`0`	`0`	`0`	`0`
`fastmem_size`	`0`	`0`	`0`	`0`	`0`	`4`
`cpu_type`	`68000`	`68000`	`68000`	`68000`	`68020`	`68040`
`fpu_model`						`68040`
`cpu_24bit_addressing`					`true`	`false`
`cpu_cycle_exact`	`true`	`true`	`true`	`true`	`true`	
`cycle_exact`	`true`	`true`	`true`	`true`	`true`	
`cachesize`	`0`	`0`	`0`	`0`	`0`	
`ntsc`	`false`	`false`	`true`	`false`	`false`	`false`
`joyport0` / `joyport1`	`mouse` / `joy0`	`mouse` / `joy0`		`mouse` / `joy0`		`mouse` / `joy0`

A blank cell means the profile leaves the setting out, so WinUAE's default applies.

Every profile also sets:

- `cpu_speed=real`, `cpu_compatible=true`;
- `warp=false`, `turbo_boot=false`;
- `gfx_framerate=1`, `gfx_linemode=double`, `sound_output=normal`;
- `joyport1_mode=djoy`, wherever the profile sets `joyport1`.

Every profile mounts `uae/dh0` read-write as `dh0:`. The debugger loads symbols from `uae/dh0/main` and runs it as `SYS:main`.

The A4000 is the only profile without cycle-exact timing, so its CPU runs well ahead of the chipset. Code that assumes the Blitter or the beam keeps pace with the CPU breaks there first (Chapter 22).

3. What the Settings Mean

Setting	Meaning
`kickstart_rom_file`	Kickstart ROM image, read from the project's `roms/` folder (`${workspaceFolder}/roms/`), or `:AROS` for the built-in replacement
`chipset`	`ocs`, `ecs` or `aga`
`chipset_compatible`	the motherboard to model: `A600` adds the A600's Gayle chip (IDE and PCMCIA)
`chipmem_size`	Chip RAM in 512 KB blocks: `1` = 512 KB, `2` = 1 MB, `4` = 2 MB
`bogomem_size`	Slow (trapdoor) RAM at `$C00000` in 256 KB blocks: `2` = 512 KB
`fastmem_size`	Fast RAM in megabytes
`cpu_type` / `fpu_model`	processor and floating-point unit to emulate
`cpu_24bit_addressing`	`true` limits the address bus to 16 MB, as on the 68000 and the A1200's 68EC020
`cpu_speed`	`real` runs the CPU at the emulated machine's speed
`cpu_cycle_exact`	cycle-exact CPU timing
`cycle_exact`	cycle-exact chipset timing; needed for accurate DMA contention and raster timing
`cachesize`	JIT cache size; `0` disables the JIT
`ntsc`	`false` = PAL 50 Hz, `true` = NTSC 60 Hz
`gfx_framerate`	`1` draws every frame, so the raster profiler and tearing are visible
`gfx_linemode`	`double` shows each line twice, the look of a real monitor
`joyport0` / `joyport1`	the device in each port
`joyport1_mode`	`djoy`: a digital joystick

4. The Build Task

The A500 PAL, Workbench and FS-UAE profiles use the `preLaunchTask` `build + ADF`, defined in `.vscode/tasks.json`: it runs `amigaassembly: build`, then `make ADF`. The other five use `amigaassembly: build` alone. The extension assembles the root `main.asm`, which includes `games/duck_rescue/main.asm` (point it at your own game to debug that instead). It runs:

```
vasmm68k_mot -m68000 -Fhunk -linedebug -ignore-mult-inc -nowarn=2047 -nowarn=2069
vlink -bamigahunk -Bstatic                          -> uae/dh0/main
```

`build.py` uses the same `vasm` flags but also passes `-s` to `vlink`, which strips the symbols. The VS Code build keeps them, so the debugger can resolve labels; that is why its executable is larger.

`make ADF` runs `python tools/make_adf.py`. It builds a stripped release copy of the current game (the one the root `main.asm` includes) into `build/adf/main`, then uses `xdftool` (from `pip install amitools`) to write a bootable floppy image, `adfs/game.adf`, holding `main`, its icon and an `S/Startup-Sequence` that runs it. The Workbench profile mounts it as `DF1:` (`nr_floppies=2`, `floppy1=${workspaceFolder}/adfs/game.adf`), so the game appears as a disk icon next to Workbench, which boots from your own image in `adfs/amiga-os-134-workbench.ad`. The same image boots on its own from `DF0:`, and can go straight onto a Gotek. Without amitools, the step prints a note and the build still succeeds. Git ignores everything in `adfs/` except its README, because a Workbench disk is copyrighted, and ignores `build/adf/`.

The `amigaassembly: build` task does **not** export levels or convert assets. After changing a map or any art, run `python tools/build.py --game duck_rescue` once, then debug as usual.